Пользовательские сценарии (Use Cases)

Проект Т-Путешествия

1. Назначение

Документ показывает, как пользователь выполняет основные действия в обязательном MVP: кто начинает сценарий, что делает система, какие возможны ошибки и чем завершается операция. Здесь описано ожидаемое поведение, а не готовность программного кода.

2. Как читать сценарии

Элемент	**Пояснение**
Участник сценария	Кто совершает действия в приложении.
Предусловие	Что должно быть выполнено до начала.
Основной сценарий	Последовательность успешных действий пользователя и системы.
Альтернативы	Что происходит, если данные ошибочны или сервис недоступен.
Результат	Что изменилось после завершения.

3. Основные пользовательские сценарии

1. **Вход и регистрация по SMS**

Поле	**Содержание**
Кто участвует	Пользователь с номером телефона
Цель	Войти в приложение без пароля
До начала	Пользователь открыл экран входа

Основной сценарий

1. Вводит номер телефона и запрашивает SMS-код.

2. Система проверяет формат номера и ограничения отправки, затем направляет одноразовый код.

3. Пользователь вводит полученный код.

4. Система проверяет код, срок действия и число попыток.

5. Если аккаунт уже существует — выполняет вход; иначе создаёт аккаунт с этим номером.

6. C# выдаёт JWT; приложение открывает личный кабинет или список поездок.

Альтернативы и ошибки

- Неверный или просроченный код — сообщение об ошибке; вход не выполняется.
- Превышен лимит попыток/отправок — повторная попытка ограничивается.
- SMS не доставлено — пользователь может запросить повторный код по правилам сервиса.

Результат: Пользователь авторизован; пароли и email для входа не используются.

2. Редактирование личного кабинета

Поле	**Содержание**
Кто участвует	Зарегистрированный пользователь
Цель	Обновить личные настройки и контактные данные
До начала	Пользователь авторизован

Основной сценарий

1. Открывает личный кабинет.

2. При необходимости указывает необязательный email, ссылку Т-Банка и выбранную тему интерфейса.

3. Нажимает «Сохранить».

4. C# проверяет допустимость и уникальность email, формат ссылки и сохраняет изменения.

5. Приложение показывает актуальные данные профиля.

Альтернативы и ошибки

- Email уже привязан к другому аккаунту — сохранить такой адрес нельзя.
- Некорректная ссылка — система показывает ошибку валидации; пользователь может исправить или очистить поле.

Результат: Изменения профиля сохранены; номер телефона остаётся обязательным идентификатором аккаунта

3. Создание поездки

Поле	**Содержание**
Кто участвует	Зарегистрированный пользователь
Цель	Создать новую поездку для общих расходов
До начала	Пользователь авторизован

Основной сценарий

1. Нажимает «Создать поездку».

2. Указывает название и диапазон дат; при необходимости обложку.

3. Подтверждает создание.

4. C# проверяет название и корректность диапазона дат.

5. Система сохраняет поездку и добавляет создателя как Owner со статусом Accepted.

6. Новая поездка появляется в списке пользователя.

Альтернативы и ошибки

- Дата завершения раньше даты начала — поездка не создаётся.
- Не заполнено название — система просит заполнить поле

Результат: Создана поездка; у автора есть права организатора.

4. Приглашение и принятие участия

Поле	**Содержание**
Кто участвует	Организатор; зарегистрированный пользователь
Цель	Подключить зарегистрированного друга к поездке
До начала	Поездка существует; организатор авторизован

Основной сценарий

1. Организатор открывает участников поездки и вводит телефон или привязанный email друга.

2. C# находит существующий аккаунт и создаёт приглашение со статусом Pending.

3. Получатель видит приглашение и выбирает «Принять» или «Отказаться».

4. Система проверяет, что решение принимает адресат приглашения.

5. При принятии статус становится Accepted и открывается доступ к поездке; при отказе — Declined

Альтернативы и ошибки

- Аккаунт с указанными контактами не найден — приглашение не создаётся.
- Повторное приглашение уже приглашённого пользователя — система не создаёт дубликат.
- Пока приглашение Pending или Declined, содержимое поездки недоступно.

Результат: Приглашение обработано; статус участия отражает выбор пользователя.

5. Добавление виртуального участника

Поле	Содержание
Кто участвует	Организатор поездки
Цель	Учитывать человека без аккаунта в расходах
До начала	Организатор авторизован; поездка доступна

Основной сценарий

1. Открывает раздел участников и выбирает «Добавить виртуального участника».

2. Указывает имя участника.

3. C# проверяет право Owner и непустое имя.

4. Система создаёт TripMember без userId, с именем и статусом Accepted.

5. Виртуальный участник становится доступен в списке плательщиков и долей.

Альтернативы и ошибки

- Недостаточно прав — добавление запрещено.
- Имя не заполнено — отображается ошибка.

Результат: Участник учитывается в поездке, но не может войти и действовать самостоятельно

6. Ручное добавление и распределение расхода

Поле	Содержание
Кто участвует	Организатор или принятый участник поездки
Цель	Зафиксировать общую трату и доли
До начала	Пользователь авторизован; имеет доступ к поездке

Основной Сценарий

1. Открывает форму «Добавить расход».

2. Указывает название, категорию, общую сумму и плательщика из участников поездки.

3. Выбирает участников, между которыми распределяется трата.

4. Выбирает «Поровну» или «Своя сумма» и задаёт доли.

5. Система проверяет принадлежность всех memberId поездке, отсутствие повторов и равенство суммы долей общей сумме.

6. После подтверждения C# сохраняет Expense и ExpenseSplits; расход появляется в ленте.

Альтернативы и ошибки

- Доли не равны сумме расхода — сохранить нельзя.
- Сумма неположительная или участник не относится к поездке — ошибка.
- Самостоятельное заполнение долей другими участниками в MVP отсутствует.

Результат: Корректный расход сохранён и учитывается в балансе

7. Распознавание чека перед сохранением расхода

Поле	**Содержание**
Кто участвует	Организатор или принятый участник поездки
Цель	Сократить ручной ввод суммы и названия
До начала	Открыта форма добавления расхода

Основной сценарий

1. Пользователь прикрепляет фотографию чека.

2. React отправляет изображение в .NET API.

3. C# проверяет входные данные и передаёт файл Python-сервису.

4. Python возвращает извлечённые поля, а C# предоставляет их как черновик.

5. Пользователь проверяет или исправляет результат, выбирает плательщика и доли.

6. Пользователь отдельно нажимает «Добавить расход», после чего применяются обычные проверки расхода.

Альтернативы и ошибки

- Неудачное распознавание — доступен ручной ввод.
- Файл не поддерживается или недоступен Python — отображается понятная ошибка.
- Результат OCR никогда не сохраняет расход автоматически.

Результат: Расход создаётся только после проверки и подтверждения пользователя.

8. Просмотр балансов и подтверждение полного долга

Поле	Содержание
Кто участвует	Зарегистрированный должник; Организатор для виртуального должника
Цель	Узнать, кому должны деньги, и отметить погашение
До начала	Поездка доступна; в ней есть расходы.

Основной сценарий

1. Пользователь открывает вкладку «Итоги».

2. C# рассчитывает сальдо по расходам, долям и ранее подтверждённым погашениям, выдаёт рекомендуемые переводы.

3. Пользователь при наличии ссылки переходит на страницу перевода Т-Банка и проводит платёж вне приложения.

4. Зарегистрированный должник нажимает «Я оплатил» для своего полного перевода; за виртуального должника действие совершает Owner.

5. C# сверяет актуальную сумму, права и защиту от повторов, затем фиксирует DebtSettled в Logs.

6. При новом расчёте подтверждённое погашение учитывается в балансе

Альтернативы и ошибки

- Частичная оплата в MVP не поддерживается.
- Нет ссылки Т-Банка — автоматического перевода нет, долг всё равно отображается.
- Долг изменился с момента отображения или запрос повторяется — приложение не должно повторно учитывать платёж.
- Подтверждение пользователя не является подтверждением банка.

Результат: Баланс отражает расходы и вручную подтверждённые погашения

9. Просмотр истории поездки

Поле	Содержание
Кто участвует	Организатор или принятый участник
Цель	Видеть изменения и зафиксированные действия
До начала	Пользователь имеет доступ к поездке

Основной сценарий

1. Открывает вкладку «Аудит (История)».

2. C# проверяет доступ и загружает записи Logs по поездке.

3. Приложение показывает хронологию изменений и подтверждений погашения.

Альтернативы и ошибки

- Нет записей — показывается пустое состояние.
- Нет доступа — история не раскрывается.

Результат: Пользователь видит историю доступной поездки

10. Импорт места или мероприятия по ссылке

Поле	**Содержание**
Кто участвует	Организатор или принятый участник
Цель	Добавить место или мероприятие в детали поездки
До начала	Пользователь авторизован и имеет доступ к поездке.

Основной сценарий

1. Открывает «Детали поездки» и вставляет ссылку на поддерживаемый источник.

2. React отправляет ссылку в C#; C# проверяет доступ пользователя.

3. Python безопасно обрабатывает ссылку и возвращает нормализованные данные.

4. C# проверяет сведения и сохраняет активность в TripActivity.

5. В таймлайне появляется импортированная запись.

Альтернативы и ошибки

- Источник не поддерживается или недоступен — импорт не выполняется.
- Невалидный ответ Python — данные не должны сохраняться частично.
- Для импорта из URL ссылка обязательна.

Результат: В таймлайне сохранено проверенное импортированное событие.

11. Поиск авиабилетов

Поле	**Содержание**
Кто участвует	Организатор или принятый участник
Цель	Посмотреть предложения перелетов для поездки
До начала	Пользователь авторизован; поездка доступна

Основной сценарий

1. В деталях поездки указывает IATA-коды отправления и назначения и дату вылета.

2. React вызывает публичный GET /api/trips/{tripId}/flights/search с JWT.

3. C# проверяет поездку и доступ, затем обращается к Python GET /internal/flights/search с внутренним API-ключом.

4. Python возвращает кэшированные предложения, отсортированные по цене в RUB.

5. C# добавляет tripId к ответу; React показывает билеты и предупреждает, что цена может измениться.

6. При наличии bookingUrl пользователь может перейти на внешний сайт продавца.

Альтернативы и ошибки

- Нет предложений — успешная пустая выдача, а не ошибка.
- Python или провайдер недоступен — понятная ошибка без раскрытия внутреннего ключа.
- Переход на внешний сайт не подтверждает покупку.

Результат: Пользователь видит результаты поиска; покупка не производится внутри приложения

12. Ручное сохранение рейса в таймлайн

Поле	**Содержание**
Кто участвует	Организатор или принятый участник
Цель	Сохранить информацию о выбранном рейсе как план
До начала	Пользователь имеет доступ к поездке и располагает сведениями о рейсе

Основной сценарий

1. Пользователь выбирает рейс из найденных предложений и нажимает «Сохранить».

2. React передаёт название, тип Flight, маршрут, при наличии точное время и sourceUrl в POST /api/trips/{tripId}/activities.

3. C# проверяет доступ, необходимые поля и допустимость предоставленной ссылки.

4. Система сохраняет запись TripActivity как запланированный перелёт.

5. В таймлайне появляется запись без отметки о подтверждённой оплате.

Альтернативы и ошибки

- bookingUrl/sourceUrl отсутствует — ручное сохранение допускается; для реализации требуется nullable SourceUrl в БД.
- Точное время рейса неизвестно — scheduledAt = null.
- Недопустимая внешняя ссылка — сохранение с такой ссылкой запрещено.

Результат: Рейс сохранён как план; расход и подтверждение покупки автоматически не создаются.